

PAM, one module at a time

The stack that decides whether you get in. Read it, call it directly, then change it seven times — password rules, lockout, limits, host restrictions, an audit hook, and a directory server. From a running lab.

Command `qlab run pam-lab`

VMs `pam-lab-server` · `pam-lab-client` · `pam-lab-ldap`

LAN `192.168.100.0/24`, isolated between the three VMs

Credentials `labuser` / `labpass` · `testuser` / `Test123!` · `alice` / `Alice123!`

Outcome `qlab test pam-lab` → 8 exercises, 43 checks, all passed

1. Three machines, one login

```
pam-lab-server      192.168.100.1/24
pam-lab-client      192.168.100.2/24
pam-lab-ldap        192.168.100.3/24
```

```
testuser:1001:/bin/bash
alice:1002:/bin/bash
```

The server is where PAM is configured. The client exists so that logins arrive over a network, from an address the server can see and make decisions about — several of the modules below only make sense when the login comes from somewhere. The LDAP server is for the last section, and is ignored until then.

Everything that follows changes the server's PAM configuration and then puts it back. That is worth saying out loud: PAM is the one subsystem where a typo locks you out of your own machine, which is why every experiment here is paired with a restore, and why `pamtester` — section 3 — exists at all.

2. The stack is a text file

```
$ cat /etc/pam.d/sshd      (comments and blank lines removed)
@include common-auth
account    required      pam_nologin.so
@include common-account
session [success=ok ignore=ignore module_unknown=ignore default=bad]      pam_selinux.so close
session    required      pam_loginuid.so
session    optional      pam_keyinit.so force revoke
@include common-session
session    optional      pam_mail.so standard noenv # [1]
session    required      pam_limits.so
session    required      pam_env.so # [1]
session    required      pam_env.so user_readenv=1 envfile=/etc/default/locale
session [success=ok ignore=ignore module_unknown=ignore default=bad]      pam_selinux.so open
@include common-password

$ the file it @includes for authentication
auth       [success=2 default=ignore] pam_unix.so nullok
auth       [success=1 default=ignore] pam_sss.so use_first_pass
auth       requisite      pam_deny.so
auth       required       pam_permit.so
auth       optional       pam_cap.so

$ and how many services on this machine have a PAM stack of their own
chfn chpasswd chsh common-account common-auth common-password common-session
common-session-noninteractive login newusers other passwd polkit-1 runuser
runuser-l sshd sssd-shadowutils su su-l sudo sudo-i
```

`/etc/pam.d/sshd` is not a config file *about* authentication; it is the authentication, in order. Each line has three parts: a **type**, a **control**, and a **module**.

The four types are four separate stacks that run at different moments. `auth` proves who you are. `account` decides whether that identity may log in *now, here* — valid, unexpired, not locked.

`session` sets up and tears down what surrounds the login. `password` runs only when a credential is being changed. A module can implement any subset of them, which is why the same `pam_unix.so` appears in several stacks doing different jobs.

The control field is the interesting one. `required`, `requisite`, `sufficient` and `optional` are the shorthand forms; `common-auth` shows the long form the shorthands expand to — `[success=2 default=ignore]` means *on success, skip the next two lines*. That is how "try local users, then try the directory, then give up" is expressed without a single `if`.

The `@include` lines matter as much as the rest: the per-service file delegates to `common-*`, which is why installing something that edits `common-auth` changes how every service on the machine authenticates.

3. Calling PAM directly

```
$ pamtester sshd testuser authenticate      - right password
pamtester: invoking pam_start(sshd, testuser, ...)
pamtester: performing operation - authenticate
Password: pamtester: successfully authenticated

$ the same call, wrong password
pamtester: invoking pam_start(sshd, testuser, ...)
pamtester: performing operation - authenticate
Password: pamtester: Authentication failure

$ and the four stacks of the same service, asked one at a time
authenticate Password: pamtester: successfully authenticated
acct_mgmt    pamtester: account management done.
open_session  pamtester: successfully opened a session
close_session pamtester: session has successfully been closed.
```

`pamtester` runs the stack the way a service would, without being one. The first call authenticates `testuser` against the `sshd` stack — no SSH daemon involved, no network, just the library. The second does the same with a wrong password and gets the failure.

The last four calls are the four types from section 2, requested one at a time against the same service. `authenticate` checks the password, `acct_mgmt` asks whether the account may be used, `open_session` and `close_session` run the session hooks. Every login you have ever performed is these four calls in that order, made by a program that then goes on to do its own job.

This is the tool that makes PAM safe to experiment with. Open a second terminal that is already logged in, change the stack, test it with `pamtester`, and only then try a real login. Editing `common-auth` and logging out to see if it worked is how people end up reaching for the rescue console.

4. Password rules that apply to some people

```
$ the rules being added to /etc/security/pwquality.conf
# == walkthrough ==
minlen = 12
minclass = 3
```

```

maxrepeat = 3
dcredit = -1
ucredit = -1
lcredit = -1
ocredit = -1

$ testuser changing his own password – three tries in one session
Changing password for testuser.
Current password: New password: BAD PASSWORD: The password contains less than 1 digits
New password: BAD PASSWORD: The password contains less than 1 digits
New password: Retype new password: passwd: password updated successfully

$ and root setting the very same weak password on that account
New password: BAD PASSWORD: The password contains less than 1 digits
Retype new password: passwd: password updated successfully
(rules removed, password restored)

```

`pam_pwquality` sits at the top of the `password` stack and vets the new password before `pam_unix` is allowed to store it. The rules are in `/etc/security/pwquality.conf`: a length floor, a minimum number of character classes, and the `*credit` settings, where a **negative** value means "require at least this many" — `dcredit = -1` is one digit, minimum.

When `testuser` changes his own password, the rules are enforced: two rejections with the reason spelled out, then a password that satisfies everything, accepted. `retry = 3` is why he gets three goes before `passwd` gives up entirely.

Then root sets the *same* rejected password on the same account, and it goes through — with the complaint printed, and ignored. `pam_pwquality` does not enforce against root unless it is given `enforce_for_root`. This surprises people regularly: a policy tested with `sudo passwd someuser` looks like it is not working at all, because for root it is only ever advice.

5. Lockout: three strikes, counted on disk

```

$ the right password, before anything is changed
login succeeded

$ the auth stack, rebuilt around pam_faillock
auth    required      pam_faillock.so preauth silent deny=3 unlock_time=300
auth    [success=3 default=ignore] pam_unix.so nullok
auth    [default=die] pam_faillock.so authfail deny=3 unlock_time=300
auth    requisite     pam_deny.so
auth    required      pam_permit.so

account required pam_faillock.so

$ three wrong passwords from the client...
attempt 1: refused
attempt 2: refused
attempt 3: refused
$ ...and then the right one
refused

$ what the server made of those four attempts
pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.100.2
user=testuser
pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.100.2
user=testuser
pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=192.168.100.2
user=testuser

```

```
pam_faillock(sshd:auth): Consecutive login failures for user testuser account temporarily locked

$ faillock --user testuser      (on the server)
testuser:
When                Type  Source                Valid
2026-09-14 14:20:49 RHOST 192.168.100.2        V
2026-09-14 14:20:52 RHOST 192.168.100.2        V
2026-09-14 14:20:55 RHOST 192.168.100.2        V

$ faillock --user testuser --reset, then the right password again
login succeeded
(stack restored)
```

The rebuilt stack shows the shape `pam_faillock` needs, and the order is not optional. `preauth` runs *before* `pam_unix` and refuses immediately if the account is already locked. `authfail` runs *after* it and records a failure — with `[default=die]`, so a failed attempt stops the stack there. Put `authfail` first and every login fails; this is a genuinely easy mistake to make.

Then three wrong passwords from the client, and the fourth attempt — with the *right* password — is refused too. The account is locked, and the server's own log says so in as many words: three `pam_unix` authentication failures, then `pam_faillock` announcing that the account is temporarily locked.

`faillock --user` prints the ledger: one row per failure, with the time and the source address, because the counter is per-user and kept in `/var/run/faillock/`. `--reset` empties it, and the same password that was refused a moment ago works again. `unlock_time=300` would have done the same thing on its own after five minutes.

6. Limits arrive with the session

```
$ limits.conf before and after, and what a login session gets
nofile for this session: 1024
pam_limits lines in common-session: 1

$ adding a per-user limit
testuser hard nofile 256
testuser soft nofile 128
testuser hard nproc 64

$ a fresh login session as testuser, from the client
128
256
64
(soft nofile, hard nofile, max user processes)
(limits.conf restored)
```

`pam_limits` is already in `common-session` — it is in almost every distribution by default — and it applies `/etc/security/limits.conf` at the moment a session opens. Nothing else in the system does this; a limit set here is a property of *logging in*, not of the shell or the user account.

Three lines produce three different answers in the new session: a soft limit, a hard limit, and a process cap. The soft one is what a program gets; the hard one is the ceiling it may raise itself to and no further. The distinction is what lets `ulimit -n 4096` sometimes work and sometimes not, on the same machine, for different users.

The measurement has to come from a *new* login. An existing shell keeps the limits it was born with, and no amount of editing `limits.conf` will change it.

7. Where you are logging in from

```
$ a deny rule in /etc/security/access.conf, and the module that reads it
-:testuser:192.168.100.2
account required pam_access.so

$ testuser logging in from 192.168.100.2 – the client
Connection closed by 192.168.100.1 port 22

$ what the server logged about it
pam_access(sshd:account): access denied for user `testuser' from `192.168.100.2'

$ the same account, logging in on the server itself
pamtester: account management done.
(access.conf and the stack restored)
```

One line of `/etc/security/access.conf` — deny, this user, from that address — and `pam_access` in the `account` stack to read it. The client's login is closed before it gets a shell, the password having been perfectly correct, and the log records exactly why.

This is the clearest demonstration in the lab of what the `account` stack is *for*. Authentication succeeded: the password was right and `pam_unix` said so. Authorisation then failed, separately, on grounds that have nothing to do with credentials. Two different questions, two different stacks, and PAM keeps them apart on purpose.

The same account still passes `acct_mgmt` when the login is not coming from the denied address — the rule names a source, so it only applies there.

8. Anything you can write in a script

```
$ a shell script bolted onto the session stack with pam_exec
session optional pam_exec.so /usr/local/bin/pam-audit.sh
#!/bin/bash
echo "$(date +%F %T)" PAM_TYPE=$PAM_TYPE USER=$PAM_USER RHOST=$PAM_RHOST SERVICE=$PAM_SERVICE
TTY=$PAM_TTY" >> /var/log/pam-audit.log

$ one SSH login from the client...
hello

$ ...and what the script wrote
2026-09-14 14:21:40 PAM_TYPE=open_session USER=testuser RHOST=192.168.100.2 SERVICE=sshd TTY=ssh
2026-09-14 14:21:40 PAM_TYPE=close_session USER=testuser RHOST=192.168.100.2 SERVICE=sshd TTY=ssh
2026-09-14 14:21:40 PAM_TYPE=open_session USER=labuser RHOST=10.0.2.2 SERVICE=sshd TTY=ssh
(sshd stack restored)
```

`pam_exec` runs an arbitrary program as part of the stack, handing it the context in environment variables. Twelve lines of shell and one line in `/etc/pam.d/sshd` produce a login audit trail: who, from where, which service, which TTY, and whether the session was opening or closing.

The log has three entries after one login, not two, and the third is the walkthrough's own SSH connection arriving to read the file. That is worth noticing — the hook is on the service, not on the user, and it sees everything the service does, including the thing you are doing right now.

`optional` is the right control here: an audit hook that can *fail the login* because a script had a typo is a bad trade. With `required`, a non-zero exit from that script would lock everyone out.

9. Users who live somewhere else

```
$ before: does this machine know who ldapuser1 is?
passwd:      files systemd sss
group:       files systemd sss
shadow:      files sss
inactive
ldapuser1:*:10001:10000:LDAP User1:/home/ldapuser1:/bin/bash

$ the directory itself, on 192.168.100.3
dn: uid=ldapuser1,ou=users,dc=pam-lab,dc=local
uid: ldapuser1
uidNumber: 10001
gidNumber: 10000
homeDirectory: /home/ldapuser1
loginShell: /bin/bash

$ pointing the server at it: /etc/sss/sss.conf
passwd:      files sss
group:       files sss
shadow:      files sss
active

$ after: the same questions, same machine
ldapuser1:*:10001:10000:LDAP User1:/home/ldapuser1:/bin/bash
uid=10001(ldapuser1) gid=10000(pamtesters) groups=10000(pamtesters)
pamtesters:*:10000:ldapuser1,ldapuser2

lines mentioning ldapuser1 in /etc/passwd: 0

$ and ldapuser1 logging in over SSH, from the client
ldapuser1
/home/ldapuser1
. . . .bash_logout .bashrc .profile
(sssd stopped, nsswitch and the session stack restored)
```

`nsswitch.conf` already lists `sss` as a source for `passwd`, `group` and `shadow` — but `sss` is not running and has no configuration, so the machine has never had anything to ask. `getent passwd ldapuser1` comes back empty.

The entry exists, one VM away, in an LDAP directory: a `uid`, a `uidNumber`, a `gidNumber`, a home directory and a login shell — the same fields `/etc/passwd` has, written as a directory object.

One config file and a restart later, and the machine answers the same questions differently. `getent` resolves the user, `id` resolves the group membership, and `/etc/passwd` still contains no trace of either. Then `ldapuser1` logs in over SSH from the client, and `pam_mkhomedir` creates the home directory on the way past, copying `/etc/skel` into it.

That is the whole point of the exercise: **identity** came from the directory via NSS, **authentication** came from the directory via `pam_sss`, and **the home directory** came from a PAM session module — three separate mechanisms that together make a user who was never created on this machine into an ordinary one.

10. Try it yourself

```
qlab run pam-lab
qlab shell pam-lab-server
```

The safe loop is always the same:

```
sudo cp /etc/pam.d/common-auth /root/common-auth.bak # first, always
sudo nano /etc/pam.d/common-auth
echo 'Test123!' | sudo pamtester -v sshd testuser authenticate
```

and from the client, a real login to confirm:

```
sshpass -p 'Test123!' ssh testuser@192.168.100.1
```

Three things worth doing:

- Change a `required` to a `requisite` in `common-auth` and watch where the stack stops — `requisite` fails immediately, `required` finishes the stack first so an attacker cannot tell *which* module refused.
- Set `deny=3` in `pam_faillock` and lock yourself out on purpose, then unlock the account from a second session with `faillock --reset`.
- Add `pam_time` alongside `pam_access` and restrict a user to office hours in `/etc/security/time.conf`.

```
Exercise 1 – PAM Anatomy
  Exercise 1: All 13 checks passed
Exercise 2 – Password Policy (pam_pwquality)
  Exercise 2: All 5 checks passed
Exercise 3 – Account Lockout (pam_faillock)
  Exercise 3: All 3 checks passed
Exercise 4 – Resource Limits (pam_limits)
  Exercise 4: All 5 checks passed
Exercise 5 – Time-based Access (pam_time)
  Exercise 5: All 2 checks passed
Exercise 6 – Host/User Access Control (pam_access)
  Exercise 6: All 3 checks passed
Exercise 7 – Custom Audit (pam_exec)
  Exercise 7: All 4 checks passed
Exercise 9 – Centralized LDAP Authentication (sssd)
  Exercise 9: All 8 checks passed
Exercises run:      8
Exercises passed:   8
Exercises failed:   0
```

`qlab test pam-lab` performs each of these changes, verifies the behaviour it was meant to produce, and restores the original files — which is also the best-documented answer to "what does this module actually do".